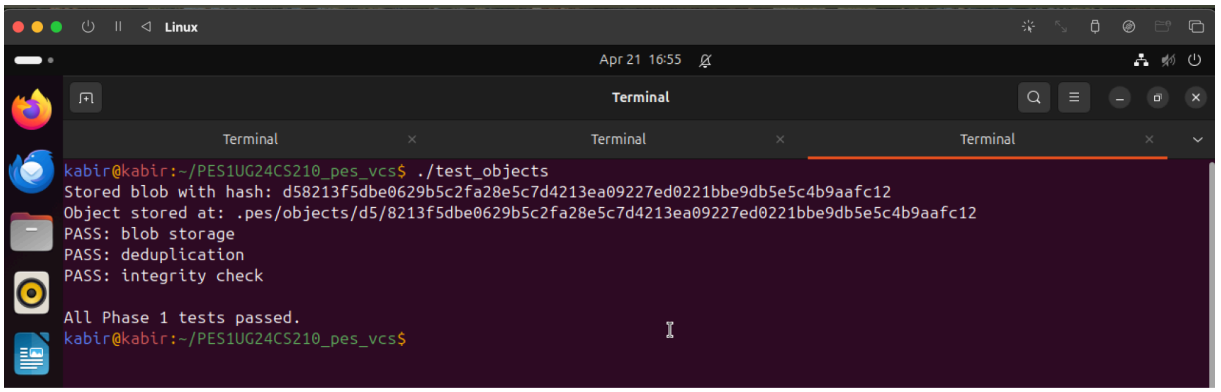
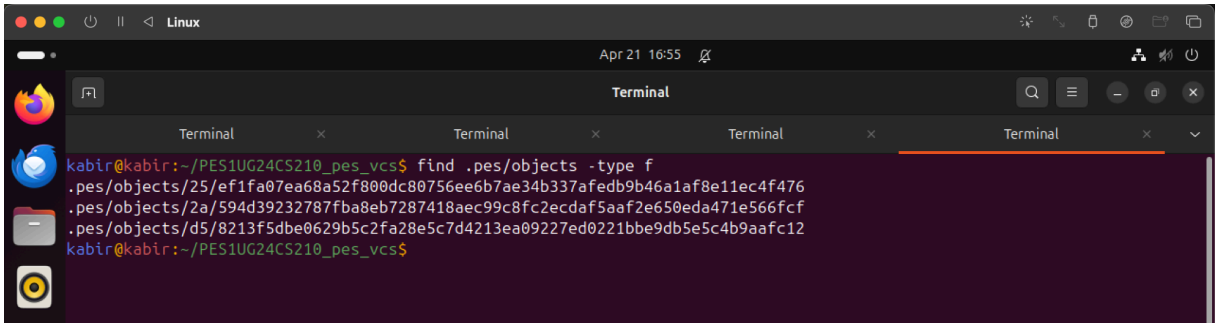
PES1UG24CS210
KABIR RAJU G

CSE – D

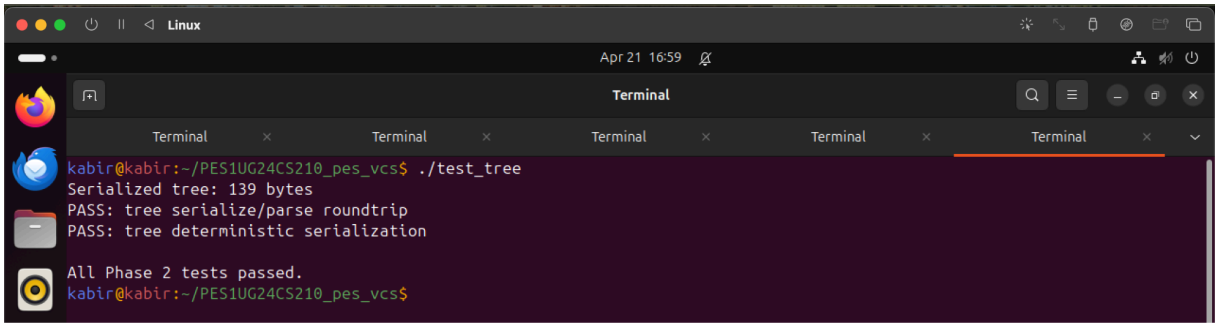
1A.

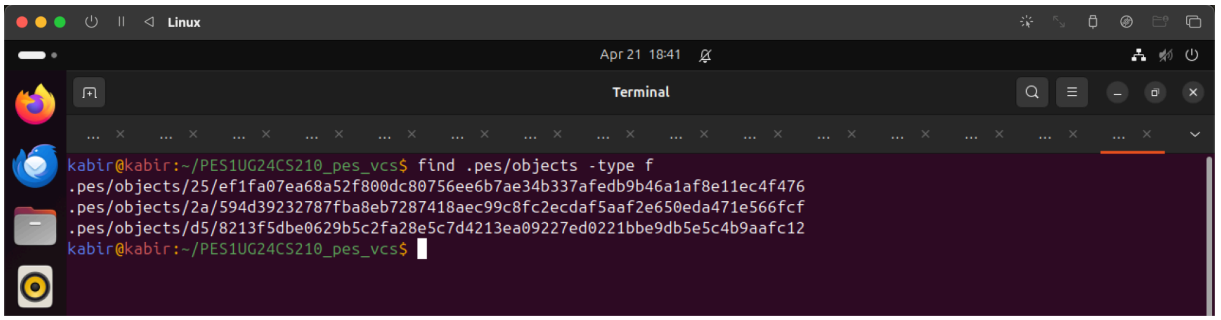


1.B



2.A





2.B

```
kabir@kabir:~/PES1UG24CS210_pes_vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
kabir@kabir:~/PES1UG24CS210_pes_vcs$
```

```
kabir@kabir:~/PES1UG24CS210_pes_vcs$ xxd .pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
| head -20
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 7479 0a                        egrity.
kabir@kabir:~/PES1UG24CS210_pes_vcs$
```

```
kabir@kabir:~/PES1UG24CS210_pes_vcs$ xxd .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
| head -20
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                      ES-VCS!.
kabir@kabir:~/PES1UG24CS210_pes_vcs$
```

3.A

```
kabir@kabir:~/PES1UG24CS210_pes_vcs$ rm -rf .pes
./pes init

echo "hello" > file1.txt
echo "world" > file2.txt

./pes add file1.txt file2.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

kabir@kabir:~/PES1UG24CS210_pes_vcs$
```

3.B

```
kabir@kabir:~/PES1UG24CS210_pes_vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776792227 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776792227 6 file2.txt
kabir@kabir:~/PES1UG24CS210_pes_vcs$
```

4.A

```
kabir@kabir:~/PES1UG24CS210_pes_vcs$ ./pes log
commit 07e7987118f047249d482f7277b896c5a7df12d54b8e52848d1c22d9f682a75d
Author: Kabir <PES1UG24CS210>
Date:   1776794591

    third commit

commit b48971eefb9135ce31bd158890365446d94c8d9a02eec3c4b817a88f1b20abd6
Author: Kabir <PES1UG24CS210>
Date:   1776794591

    second commit

commit 15cee6699039272717f422250f82c199b2e3c3e55741ab5e799921c0fa249d4f
Author: Kabir <PES1UG24CS210>
Date:   1776794591

    first commitira

kabir@kabir:~/PES1UG24CS210_pes_vcs$
```

4.B

```
kabir@kabir:~/PES1UG24CS210_pes_vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/07/e7987118f047249d482f7277b896c5a7df12d54b8e52848d1c22d9f682a75d
.pes/objects/09/b576c2eb51b78fcf9b8493a0909522a75f8ee86df0e926312deeb5fbe7dfb1
.pes/objects/15/cee6699039272717f422250f82c199b2e3c3e55741ab5e799921c0fa249d4f
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/a5/bbe3d6651489c9bdb081dc4d0946f06c138864b9f6fed8c3f0105062691b69
.pes/objects/b4/8971eefb9135ce31bd158890365446d94c8d9a02eec3c4b817a88f1b20abd6
.pes/objects/bf/04cdec355873de29e19b0eedeffe96e3a8626dce69ac6bb6ee80682bf44e2a
.pes/objects/d1/191de3b7f7574404e0eb7e3869497e454112feb09527b93d1da0cb53169c85
.pes/objects/fe/76325aa5521b207ebe01e12fd8e9e3abf030cacd5398e3744a3a56a81ad1bd
.pes/refs/heads/main
kabir@kabir:~/PES1UG24CS210_pes_vcs$
```

4.C

```
kabir@kabir:~/PES1UG24CS210_pes_vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
07e7987118f047249d482f7277b896c5a7df12d54b8e52848d1c22d9f682a75d
ref: refs/heads/main
kabir@kabir:~/PES1UG24CS210_pes_vcs$
```

= Running integration tests =

= PES-VCS Integration Test =

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists

```
PASS: .pes/HEAD exists

--- Staging Files ---
Added file(s) successfully
Status after add:
Staged changes:
  staged: file.txt
  staged: hello.txt

--- First Commit ---
Committed: 8d442ed87ce92af985391cb7893a81d88686e13e34fbe3189ab5593033d38b3b
Initial commit

--- Second Commit ---
Committed: 11e5bbd01d382cfcd9d64f5f2ed20a7771b5174469f5dcd947847ddf6699b9a6
Update file.txt

--- Third Commit ---
Committed: 458b664feede2679893cb060489d0be7eb112c70b351396a5287a9a22b1ce6d5
Add farewell

--- Full History ---
commit 458b664feede2679893cb060489d0be7eb112c70b351396a5287a9a22b1ce6d5
Author: PES User <pes@localhost>
Date: 1776787869
Add farewell

commit 11e5bbd01d382cfcd9d64f5f2ed20a7771b5174469f5dcd947847ddf6699b9a6
Author: PES User <pes@localhost>
Date: 1776787869
Update file.txt

commit 8d442ed87ce92af985391cb7893a81d88686e13e34fbe3189ab5593033d38b3b
Author: PES User <pes@localhost>
Date: 1776787869
Initial commit

--- Reference Chain ---
HEAD:
ref: refs/heads/main

refs/heads/main:
458b664feede2679893cb060489d0be7eb112c70b351396a5287a9a22b1ce6d5

--- Object Store ---
Total objects created: 10
```

```
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/11/e5bbd01d382cfcd9d64f5f2ed20a7771b5174469f5dcd947847ddf6699b9a6
.pes/objects/45/8b664feede2679893cb060489d0be7eb112c70b351396a5287a9a22b1ce6d5
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/8d/442ed87ce92af985391cb7893a81d88686e13e34fbe3189ab5593033d38b3b
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

= All integration tests completed =
```

Phase 5 s 6 — Analysis Answers

Q5.1 — How would you implement pes checkout <branch>?

A branch is just a file in .pes/refs/heads/<branch> containing a commit hash.

What files must change

1. .pes/HEAD
    - Update to:
    - ref: refs/heads/<branch>
2. .pes/refs/heads/<branch>
    - Already exists (points to commit)

What must happen to working directory

1. Read target commit hash from branch file
2. Load commit → get tree hash
3. Traverse tree:
    - Recreate files in working directory
    - Overwrite existing tracked files

4. Remove files that exist in current branch but not in target branch
5. Update .pes/index to match new tree

Why this is complex

Because you are syncing three states:

working directory ↔ index ↔ object store (tree)

Also:

- Must avoid overwriting user changes
- Must correctly delete extra files
- Must reconstruct directory structure recursively

Q5.2 — Detecting dirty working directory conflict

Goal

Prevent checkout if:

working file ≠ index version AND branch version ≠ index version

How to detect

For each tracked file:

1. From index → get stored hash
2. From working directory → recompute hash
3. Compare:
    - If different → file is modified

Then:
4. Load target branch tree → get file hash
5. Compare:

- If working file ≠ index AND target ≠ index → conflict

Condition for refusal

modified_in_working_dir AND different_in_target_branch → abort checkout

---

Q5.3 — Detached HEAD behavior

What it means

.pes/HEAD contains:

<commit-hash>

instead of:

ref: refs/heads/main

What happens when committing

- New commits are created
- But no branch points to them
- They become **unreachable from any**

branch Risk

Commits can be lost (garbage collected later)

Recovery methods

1. Create branch at that commit:
2. pes branch new-branch
3. Use commit hash manually:
4. pes checkout <hash>
5. (In real Git: reflog helps)

Q6.1 — Garbage collection algorithm

Goal

Delete objects not reachable from any branch

Algorithm

1. Initialize:
2. reachable = empty set
3. Start from all branch heads:
    - .pes/refs/heads/*
4. For each commit:
    - Mark commit hash
    - Traverse:
        - parent commit
        - tree
5. For each tree:
    - Mark tree hash
    - Traverse:
        - blobs

        - subtrees

6. After traversal:
    - All visited = reachable
7. Scan .pes/objects/
    - If object NOT in reachable → delete

Data structure

Hash set (O(1) lookup)

Scale estimation

Given:

- 100,000 commits
- 50 branches

Worst case:

Visit ≈ 100,000 commits

+ trees (≈ same order)

+ blobs (depends, but large)

~200,000 – 500,000 objects

Q6.2 — Why GC + commit is dangerous

Race condition scenario

1. Commit process:
    - creates tree
    - creates commit object
    - (NOT yet updated HEAD)
2. GC runs simultaneously:
    - scans reachable objects from HEAD
    - new commit is NOT reachable yet
3. GC deletes:

newly created objects (tree/commit)

4. Commit finishes:
    - updates HEAD → points to deleted objec

Result

Repository corruption

How real Git avoids this

Techniques used

1. **Object immutability**
    - objects never modified
2. **Reference-based protection**
    - GC only deletes objects not referenced by:
        - branches
        - tags
        - reflog
3. **Reflog safety**
    - keeps history of HEAD changes
    - prevents recent commits from deletion
4. **Locking / atomic updates**
    - commit updates HEAD atomically
    - GC respects locks
5. **Grace period**

Objects are not deleted immediately